

This problem set covers heaps, BSTs, and AVL trees. You can use data structures from class as builtins. On some questions it may be helpful to modify one of these data structures (by storing an additional value, for example). You do not have to give full pseudocode for the modified data structure but you should give enough pseudocode that you can support your runtime claims.

The points don't correspond to the difficulty. A star after the point value indicates that I think the question is harder than the others.

Q1. [8] Give pseudocode for a function `tree-range(bst, min, max)` that returns the sum of all values in `bst` between `min` and `max`, inclusive.

Your algorithm should run in $O(n)$ time. We may deduct points for inefficiency even if it doesn't impact the asymptotic runtime - you should visit as few nodes in `bst` as possible. Note: You can't modify the data structure to store additional information, and `tree-range` must operate on a "standard" BST.

- (a) [4] **Pseudocode**
- (b) [4] **Prove that your algorithm visits all nodes with values between `min` and `max`.**

Solution

- (a) **Pseudocode**

```
tree-range(bst, min, max):
    return range-sum-helper(bst.root, min, max)

range-sum-helper(node, min, max):
    if node == null:
        return 0

    sum = 0

    // Only explore left subtree if current value > min
    if node.value > min:
        sum = sum + range-sum-helper(node.left, min, max)

    // Include current node if it's in range
    if node.value >= min and node.value <= max:
        sum = sum + node.value

    // Only explore right subtree if current value < max
    if node.value < max:
```

```

        sum = sum + range-sum-helper(node.right, min, max)

    return sum

```

(b) **Proof Of Algorithm Correctness**

Claim: The algorithm visits all nodes with values in $[\text{min}, \text{max}]$, and only visits the nodes necessary to find them. We can prove this by establishing three key properties based on the requirements of BST and our Algorithm structure.

Part 1: All in-range nodes are visited

Consider any node v with $\text{min} \leq v.\text{value} \leq \text{max}$. We show that our algorithm will visit node v by examining the path from the root to node v . For any ancestor a along this path, exactly one of two cases holds:

- If v is in a 's left subtree: By the BST property, $a.\text{value} > v.\text{value}$. Since $v.\text{value} \geq \text{min}$, we have $a.\text{value} > \text{min}$, which triggers our condition to explore $a.\text{left}$.
- If v is in a 's right subtree: By the BST property, $a.\text{value} < v.\text{value}$. Since $v.\text{value} \leq \text{max}$, we have $a.\text{value} < \text{max}$, which triggers our condition to explore $a.\text{right}$.

Thus by induction on the depth of v , we recursively explore the correct child at each ancestor, eventually reaching v itself. And when we do reach v , the condition $\text{min} \leq \text{node.value} \leq \text{max}$ is satisfied, so we add $v.\text{value}$ to our sum.

Part 2: Each in-range node is counted exactly once

From Part 1, we visit every node v such that $\text{min} \leq v.\text{value} \leq \text{max}$. This explicit range check ensures that each valid node's value is added to the sum exactly once. The recursive structure of the traversal also prevents any double-counting.

Part 3: The Algorithm Is Efficient

This algorithm ignores entire subtrees when they cannot contain in-range values:

- *Left subtree pruning:* We only explore the left child when $\text{node.value} > \text{min}$. If instead $\text{node.value} \leq \text{min}$, then by the BST property, all nodes in the left subtree have values $< \text{node.value} \leq \text{min}$, placing them outside our range.
- *Right subtree pruning:* We only explore the right child when $\text{node.value} < \text{max}$. If instead $\text{node.value} \geq \text{max}$, then by the BST property, all nodes in the right subtree have values $> \text{node.value} \geq \text{max}$, placing them outside our range.

Thus this pruning strategy ensures we avoid visiting subtrees that provably contain no in-range values.

Runtime Analysis: The algorithm runs in $O(k+h)$ time, where k is the number of in-range nodes and h is the tree height. We visit each of the k in-range nodes exactly once, plus at most $O(h)$ additional nodes along paths that lead to pruned/ignored subtrees. Thus in the worst case where all n nodes fall within the range, this gives $O(n)$ time as required. $\square$

Q2. [8] Design a data structure that supports two operations, insert and find-median.

Insert(x) should run in $O(\log n)$ time, where n is the number of items that have been inserted. **Find-median** should return the $\lceil n/4 \rceil^{\text{th}}$ largest item that has been inserted, and should run in $O(1)$ time.

Note: If you want, pretend it doesn't say "find-median", and instead "find-lower-quartile".

- (a) [4] **Pseudocode**
- (b) [4] **Runtime analysis**

Solution

(a) **Data Structure Design and Pseudocode**

Data Structure:

Let's maintain two heaps that partition the inserted elements:

- **max_heap:** A max-heap containing the smallest $\lceil n/4 \rceil$ elements
- **min_heap:** A min-heap containing the remaining (larger) elements
- **n:** Total number of elements inserted

Explanation: An invariant is a property that remains true throughout the algorithm's execution. Here our invariant is: the **max_heap** contains exactly $\lceil n/4 \rceil$ elements (the smallest ones), and all elements in **max_heap** are $\leq$ all elements in **min_heap**. In this way, the maximum of **max_heap** will always be the lower quartile.

Pseudocode:

```
initialize():
    max_heap = new MaxHeap()
    min_heap = new MinHeap()
    n = 0

insert(x):
    n = n + 1
    target_size = ceil(n / 4)
```

```
// Insert into appropriate heap
if max_heap.size() == 0 or x <= max_heap.peek():
    max_heap.insert(x)
else:
    min_heap.insert(x)

// Rebalance to maintain invariant
while max_heap.size() > target_size:
    element = max_heap.extract_max()
    min_heap.insert(element)

while max_heap.size() < target_size and min_heap.size() > 0:
    element = min_heap.extract_min()
    max_heap.insert(element)

find-median():
    return max_heap.peek()
```

(b) **Runtime Analysis**

Analysis of find-median():

This operation simply returns the maximum element from `max_heap` using the `peek()` operation; and since peeking at the top of a heap requires no comparisons or structural changes, it runs in $O(1)$ time.

Analysis of insert(x):

Let's analyze each component of the insertion operation separately:

Step 1: Update counter and compute target size

Computing `n = n + 1` and `target_size = ceil(n/4)` are simple operations that take $O(1)$ time.

Step 2: Initial insertion

We insert x into either `max_heap` or `min_heap` based on its comparison with `max_heap.peek()`. This insertion may require bubbling up the element through at most $O(\log n)$ levels of the heap.

Step 3: Rebalancing

When n increases by 1, the target size changes by at most 1:

$$\lceil n/4 \rceil - \lceil (n-1)/4 \rceil \in \{0, 1\}.$$

Hence, to restore the invariant after the initial insertion, we need to move at most one element between the heaps. Concretely, either move the max of `max_heap` to `min_heap`, or move the min of `min_heap` to `max_heap`. And since each move does one extract and one insert, costing $O(\log n)$, rebalancing is also $O(\log n)$.

Overall insert runtime:

$$O(1) + O(\log n) + O(\log n) = O(\log n)$$

Summary: By maintaining the invariant that `max_heap` contains exactly the $\lceil n/4 \rceil$ smallest elements, its maximum element is precisely the $\lceil n/4 \rceil$ -th smallest element. This corresponds to the $(n - \lceil n/4 \rceil + 1)$ -th largest element, which is the lower quartile as required.

Q3. [16] Design and implement a data structure that supports two operations, insert and kmin.

`Insert` and `kmin` should run in $O(\log n)$ time, where n is the number of items that have been inserted. `kmin` should return the i^{th} smallest item that has been inserted, where i is the number of times `kmin` has been called.

You will be tested only on the `solve_kmins` function. `solve_kmins` takes a list of commands as input. A command is a string, either “insert ” followed by a number, or “kmin”. You will not be tested on any other inputs. Output should be a list containing the return values of each call to `kmin`.

For example, `solve_kmins(["insert 9", "insert 3", "kmin", "kmin", "insert 1", "insert 12", "kmin", "kmin"])` should return `[3, 9, 9, 12]`.

- (a) [4] Give pseudocode for your data structure.
- (b) [12] Code it up and submit in `kmins.py`. Some starter code is provided; you will have to implement any data structures yourself.

Solution

(a) **Data Structure Design and Pseudocode**

Pseudocode Summary: We use an augmented AVL tree, like the Rank example shown in lecture, in which every node stores `value`, child pointers `left` and `right`, `height` (for AVL balancing) and `size` (the number of nodes in the subtree). On `insert`, update the node’s `height` and `size` and perform AVL rotations as needed. To answer the i -th smallest query, let L be the size of the left subtree. If $i \leq L$, recurse into the left subtree;

if $i = L + 1$, return the current node; otherwise recurse into the right subtree with index $i - (L + 1)$.

Pseudocode:

```
Class AVLNode:
    value, left, right, height, size

Class KMinDataStructure:
    root = None
    kmin_count = 0

    insert(value):
        root = insert_helper(root, value)

    insert_helper(node, value):
        if node is None:
            return new AVLNode(value)

        if value < node.value:
            node.left = insert_helper(node.left, value)
        else:
            node.right = insert_helper(node.right, value)

        update_height_and_size(node)
        return rebalance(node)

    kmin():
        kmin_count += 1
        return find_kth_smallest(root, kmin_count)

    find_kth_smallest(node, k):
        left_size = size(node.left)

        if k == left_size + 1:
            return node.value
        else if k <= left_size:
            return find_kth_smallest(node.left, k)
        else:
            return find_kth_smallest(node.right, k - left_size - 1)
```

Runtime Analysis:

insert(value): This performs normal AVL tree insertion with rotations to maintain balance. The insertion goes from root to a leaf position in $O(\log n)$ time, then performs at most $O(\log n)$ rotations during the rebalancing phase as we bubble back up. And since updating the size field at each node along the path only takes $O(1)$ time per node, the total runtime is $O(\log n)$.

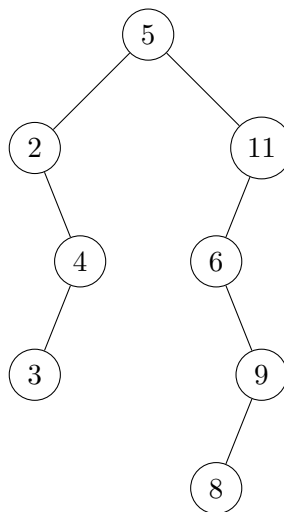
kmin(): To find the k -th smallest element, we use the augmented size information. At each node, we compare k with the size of the left subtree to determine which subtree contains the k -th smallest element. This comparison takes $O(1)$ time, and we recurse down at most $O(\log n)$ levels (the height of the AVL tree).

Overall, the BST property ensures elements are sorted. At any node with left subtree of size s , the node itself is the $(s + 1)$ -th smallest in its subtree. By recursively comparing k with left subtree sizes, we efficiently locate the k -th smallest element.

(b) **Implementation**

See `kmins.py` for the complete implementation. The solution uses an augmented AVL tree with standard AVL rotations to maintain balance, stores subtree sizes at each node to support $O(\log n)$ k -th smallest queries, and maintains a counter to track the number of `kmin` calls.

Q4. [6] What's the minimum number of rotations needed to make this an AVL tree (i.e. have balance factor between 1 and -1 for all nodes)?



Prove it by:

- (a) **[2] Give a sequence of rotations that AVL-ifies the tree (you can just list them like “5 R, 3 L, ...” for right rotation at the node containing 5 then left rotation at the node containing 3).**

- (b) [4*] Prove it's not possible to balance with fewer rotations.

Solution

- (a) Sequence of Rotations

Initial Analysis:

Using $BF(v) = h(\text{right}) - h(\text{left})$, an AVL tree requires $-1 \leq BF \leq 1$ at every node. Computing balance factors for the given tree, the only violations are:

- Node 2: $BF = 2$
- Node 6: $BF = 2$
- Node 11: $BF = -3$

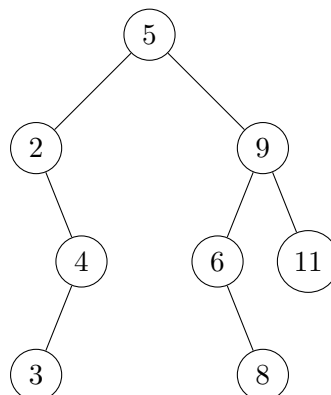
All other nodes have balance factors in $\{-1, 0, 1\}$. Thus, there are **three imbalance points** to be fixed.

Rotation Sequence (4 rotations):

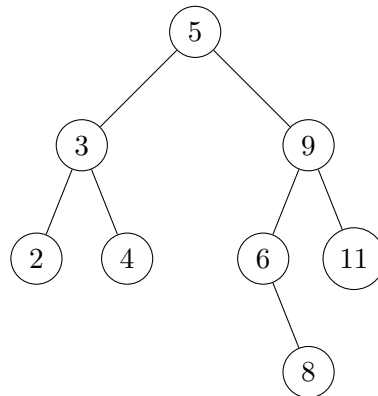
- i. **6 L** (Left rotation at 6)
- ii. **11 R** (Right rotation at 11)
- iii. **4 R** (Right rotation at 4)
- iv. **2 L** (Left rotation at 2)

Step-By-Step Transformation:

After rotations 1–2 (fixing the right subtree):



After rotations 3–4 (fixing the left subtree):



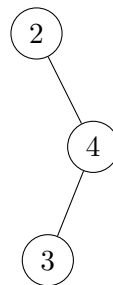
All nodes now have balance factors in $\{-1, 0, 1\}$

(b) **Proof That 4 Rotations Is Minimum**

Claim: At least 4 rotations are required to balance this tree.

Part 1: The left subtree requires at least 2 rotations.

The subtree rooted at node 2 has the structure:



This is a **Right-Left (RL) case** where node 2 is right-heavy ($BF = 2$) and node 2's right child (4) is left-heavy ($BF = -1$).

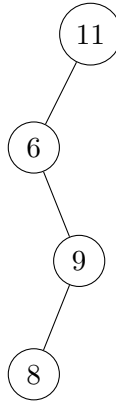
By AVL rotation rules, a single rotation can only occur at a node with $|BF| \geq 2$ or at its heavy child. Thus, the only possible single rotations are at node 2 or at node 4. So to check if we can fix this with just 1 rotation, let's check all possibilities:

- *Only R at 4:* Results in the chain $2 \rightarrow 3 \rightarrow 4$, leaving node 2 with $BF = 2$ (still violated)
- *Only L at 2:* Results in the chain $4 \rightarrow 2 \rightarrow 3$, giving node 4 $BF = -2$ (new violation)

A single rotation either preserves the original violation or creates a new one. The RL case is well-known to **require exactly 2 rotations** (R at the child, then L at the parent).

Part 2: The right subtree requires at least 2 rotations.

The subtree rooted at node 11 has violations at both nodes 6 and 11:



By AVL rotation rules, a single rotation can only occur at a node with $|\text{BF}| \geq 2$ or at its heavy child. In this subtree, such nodes are node 11, its heavy child 6, and 6's heavy child 9. Thus, to determine whether one rotation suffices, we consider the only 3 possible single rotations:

- *R at 11*: Creates the structure $6 \rightarrow 11 \rightarrow 9 \rightarrow 8$, giving node 6 $\text{BF} = 3$ and node 11 $\text{BF} = -2$ (both violated)
- *L at 6*: Fixes node 6 but leaves node 11 with $\text{BF} = -3$ (still violated)
- *R at 9*: Does not reduce the height of the subtree rooted at 6, so node 6 remains unbalanced and the violation at 11 is unaffected

Thus, no single rotation eliminates both violations; so we need **at least 2 rotations**.

Part 3: Rotations in disjoint subtrees are independent.

Rotations only restructure the subtree rooted at the rotation point. Since the left subtree (containing nodes 2, 3, 4) and the right subtree (containing nodes 6, 8, 9, 11) are completely disjoint and share no nodes, rotations in one subtree cannot affect balance factors in the other.

Conclusion:

Hence, by the independence of disjoint subtrees (Part 3), the minimum number of rotations needed is:

$$(\text{minimum for left subtree}) + (\text{minimum for right subtree}) = 2 + 2 = \boxed{4 \text{ rotations}} \quad \square$$